

Ömrüm Ceren GÜLER

Software Engineering Graduate

Adana, Turkey | +90 530 366 0135 | omrumguler35@gmail.com

<https://www.linkedin.com/in/cerengulerr>

<https://github.com/omrumcerenguler>

Internship Projects

Çukurova University, IT Department - Software Engineering Summer Intern
Adana/Turkey (July 2025 - September 2025)

1) **FastAPI Data Analytics & Prediction Service** | Backend Developer

Internship Project – Çukurova University

- **Project Overview:** Developed a FastAPI-based service application for Çukurova University to provide data analysis, predictive modeling, and various API endpoints.
- **Technical Implementation:** Established secure connections to SQL Server databases using SQLAlchemy and pyodbc for robust data retrieval.
- **Analytics & Forecasting:** Integrated Prophet and Linear Regression models to perform advanced data analysis and forecasting tasks.
- **System Management:** Implemented Swagger (OpenAPI) for interactive API testing and utilized .env files for secure management of sensitive database credentials.
- **Tools Used:** FastAPI, SQL Server, SQLAlchemy, Prophet, Linear Regression, Swagger UI.

2) **Automated Passport Photo Compliance System** | Computer Vision Developer

Internship Project – Çukurova University

- **Project Overview:** Built an automated API and bulk control tool to evaluate the compliance of ID and passport photos with official government standards.
- **Computer Vision Logic:** Leveraged MediaPipe and OpenCV for face detection, background analysis, and meticulous checks for contrast, color, and lighting.
- **Validation Features:** Engineered modules to verify image resolution, aspect ratios, EXIF data, and rotation, including optional accessory detection (glasses, hats, etc.).
- **Scalability & Security:** Developed a dedicated CLI client for bulk testing and implemented rate limiting to ensure API security and stability.
- **Tools Used:** FastAPI, MediaPipe, OpenCV, Python (CLI Client), Image Processing Techniques.

3) **Reservation Segmentation & Analytics System** | Data Analyst & Developer

Internship Project – Çukurova University

- **Project Overview:** Developed a Python-based analytical system to process corporate reservation data stored in SQL Server, dividing them into meaningful segments for business insights.

- **Machine Learning Implementation:** Applied the K-Means clustering algorithm to automatically group reservations into short-term and long-term categories based on calculated durations.
- **Predictive Classification:** Implemented a Random Forest classification model to predict the segment of new reservation entries based on time and location data.
- **Data Engineering & Analysis:** Performed time-based density analysis to identify peak hourly and daily reservation patterns and utilized Pandas for complex duration calculations.
- **Visualization & Security:** Generated statistical distribution charts and visual summaries using Matplotlib and secured database access credentials using .env files.
- **Tools Used:** Python, SQL Server, Scikit-Learn (K-Means, Random Forest), Pandas, PyODBC, Matplotlib, python-dotenv.

4) Academic Publication Analysis & Forecasting System | Data Analyst & Developer

Internship Project – Çukurova University

- **Project Overview:** Developed an analytical system for Çukurova University to evaluate academic publication trends and generate data-driven forecasts for the 2025 academic year.
- **Statistical Modeling:** Conducted rigorous model comparison studies between Linear Regression and Simple Average methods to determine optimal forecasting accuracy for unit-based and author-based metrics.
- **Data Engineering:** Executed complex data extraction and processing from SQL Server relational databases, managing multi-table schemas including author records and research area attributes.
- **Analytical Insights:** Identified top-performing academic units and the fastest-growing research domains by analyzing past publication trends and unit-specific popularity.
- **Visualization & Security:** Engineered automated reporting modules using ggplot2 and showtext for high-quality visual summaries, while securing database credentials through dotenv.
- **Tools Used:** R (dplyr, ggplot2, DBI, odbc, dotenv, showtext), SQL Server.

5) Academic Publication Prediction & Analysis System | Data Scientist & Developer

Internship Project – Çukurova University

- **Project Overview:** Developed a specialized analytical system to forecast future academic publication counts by research area, leveraging historical multi-year datasets.
- **Forecasting Strategies:** Implemented dual-modeling approaches using Facebook Prophet for advanced time-series forecasting and Linear Regression (utilizing numpy.polyfit) for comparative trend analysis.
- **Data Engineering:** Engineered direct data extraction pipelines from SQL Server utilizing SQLAlchemy for efficient retrieval and Pandas for complex data processing.
- **Interactive Interface:** Designed an interactive terminal menu allowing users to filter data by academic field, select specific prediction years, and switch between forecasting models dynamically.
- **Visual Analytics:** Automated the generation of graphical outputs using Matplotlib and Seaborn to visualize yearly publication trends and predictive comparisons.
- **Security & Environment:** Utilized python-dotenv to securely manage environment variables and sensitive database connection credentials.
- **Tools Used:** Python (3.8+), Facebook Prophet, Numpy, Pandas, SQLAlchemy, Matplotlib, Seaborn, SQL Server.

6) Academic Success Classification & Performance Analysis System | Data Scientist & Developer

Internship Project – Çukurova University

- **Project Overview:** Developed a comprehensive performance analysis system for Çukurova University to evaluate and classify the achievement levels of academic units using the STRATEJİK_PLAN database.
- **Data Science Workflow:** Executed a full-cycle data pipeline including direct data extraction from SQL databases, rigorous preprocessing, and classification logic based on targeted versus actual performance ratios.
- **Security & Privacy:** Implemented industry-standard security practices by managing database credentials through .env files and ensuring data privacy by utilizing anonymized datasets for testing.
- **System Architecture:** Designed a clean and maintainable project structure utilizing .gitignore for credential protection and requirements.txt for robust dependency management.
- **Automated Testing:** Developed specialized Python scripts for generating synthetic data samples to validate classification logic and system stability without compromising sensitive information.
- **Tools Used:** Python, SQL, Pandas, NumPy, python-dotenv, SQL Server.

7) YÖKSİS Unit Hierarchy Visualization | Data Visualization Developer

Internship Project – Çukurova University

- **Project Overview:** Developed an interactive treemap visualization tool to map the organizational hierarchy (faculties, institutes, departments) of Çukurova University using official YÖKSİS datasets.
- **Interactive Interface:** Engineered a dynamic dashboard using Streamlit and Plotly Express, enabling users to navigate seamlessly through multi-level hierarchies (Faculty Department Sub-unit).
- **Data Exploration Features:** Implemented advanced filtering logic including real-time search bars, active-unit filtering, and a "reset zoom" functionality for enhanced data discovery.
- **Backend & Security:** Established high-performance database connections utilizing SQLAlchemy and PyODBC, while managing sensitive credentials securely through .env and .gitignore practices.
- **UX Optimization:** Integrated Watchdog for performance monitoring and implemented detailed hover-over metadata displays to provide granular information without cluttering the main visual.
- **Tools Used:** Python 3.13, Streamlit, Plotly Express, Pandas, SQLAlchemy, PyODBC, Watchdog.

8) University Publication Analysis Dashboard | Data Scientist & Developer

Internship Project – Çukurova University

- **Project Overview:** Engineered a Streamlit-based web application designed to analyze and visualize academic publication data, providing users with the ability to generate real-time insights from SQL Server or offline CSV datasets.
- **Data Integration:** Built a flexible data sourcing module that executes real-time queries from Microsoft SQL Server while supporting fallback CSV uploads for offline system validation and testing.
- **Analytical Modeling:** Implemented K-Means Clustering to categorize similar academic units and utilized Facebook Prophet for long-term forecasting of future publication trends.
- **Interactive Visualization:** Designed multi-dimensional dashboards featuring line charts, bar graphs, and treemaps for comparative analysis across different academic fields and departments.

- **Granular Exploration:** Developed dynamic filtering systems that allow users to drill down into trends by specific year, research area, or academic unit for multi-level business intelligence.
- **Tools Used:** Python, Streamlit, MS SQL Server, K-Means (Scikit-Learn), Facebook Prophet, Pandas, Data Visualization (Charts & Treemaps).

9) **Academic Publication Forecasting & Hierarchy Visualizer** | Data Scientist & Developer

Internship Project – Çukurova University

- **Project Overview:** Developed a Streamlit-based web application for Çukurova University to provide integrated academic publication analysis, forecasting, and organizational unit hierarchy mapping.
- **Hierarchical Exploration:** Engineered a dynamic selection system for navigating complex academic structures, enabling users to drill down from faculties to specific departments and sub-units based on YÖKSİS IDs.
- **Forecasting Engine:** Implemented predictive modeling modules utilizing Facebook Prophet and Linear Regression to generate future publication estimates based on historical data trends.
- **Visual Analytics:** Designed interactive treemap visualizations and yearly trend charts to simplify the analysis of publication densities across various academic fields.
- **Functional Flexibility:** Integrated customizable year-range selections and the option for single-year predictions, supporting both root-level and sub-unit inclusive data aggregation.
- **Tools Used:** Python, Streamlit, SQL Server, Facebook Prophet, Linear Regression, Treemap Visualization.

Work Samples

1) **Online KIBHAS Ticket Reservation System** | Database Developer

Academic Project – Software Planning and Management | March 2025 – April 2025

- **Project Overview:** Architected a comprehensive software project plan and architectural design for a web-based bus reservation system aimed at digitizing transportation services in Cyprus.
- **Database Design & Architecture:** Designed a relational database schema and Entity-Relationship (ER) diagrams tailored for high-frequency transactions, including seat availability, passenger records, and automated booking confirmations.
- **Query & Logic Development:** Modeled data classification structures and planned optimized SQL query modules for real-time seat selection and dynamic ticket pricing.
- **Technical Planning:** Conducted technical feasibility studies for integrating PostgreSQL with a Node.js backend, ensuring data integrity and scalability.
- **Security & Compliance:** Defined data architecture standards following GDPR for personal data protection and PCI-DSS for secure payment processing simulations.
- **SDLC & Methodology:** Utilized Agile (Scrum) methodology and the Evolutionary Development Model to manage the project lifecycle, from requirement analysis to testing phase planning.
- **Tools Used:** MS Visio (System Modeling), MS Project (Resource Scheduling), Figma (UI/UX workflow), and Modelio (UML).

2) **Dynamic Weather Prediction & Smart Farming System** | System Architect

Academic Project – Software Engineering Management | October 2025 – December 2025

- **Project Overview:** Designed a smart farming decision-making assistant that utilizes real-time weather forecasting and IoT sensor data to provide actionable recommendations for irrigation and planting.
- **System Architecture:** Architected a layered modular architecture comprising a Data Collection Layer, AI Processing Core, and a Physical Field Layer for autonomous control.
- **Technical Solutions:** Designed a "Data Processing Layer" middleware to resolve data normalization challenges by standardizing disparate API and sensor inputs into a unified JSON format.
- **IoT & Automation Logic:** Modeled a smart irrigation loop featuring soil moisture analysis and "Virtual Actuators" to simulate automated valve control within a 5-second hardware response constraint.
- **Tools Used:** UML (Use Case & Class Diagrams), API Gateway, IoT Sensor Simulation, and Map-based Field Management interfaces.

3) **Community Event Management System for TRNC** | Project Manager / Developer

Academic Project – Software Project Planning and Management | September 2025 – January 2026

- **Project Overview:** Architected a comprehensive software project plan for a centralized web-based platform to manage and publish municipal events (festivals, workshops, etc.) across TRNC.
- **Analytical Planning:** Conducted root-cause analysis using Fishbone (Ishikawa) Diagrams and prioritized user pain points through Pareto Analysis to define core system requirements.
- **Feasibility & Estimation:** Performed rigorous effort estimation using the COCOMO model (67.5 person-months) and established project timelines via PERT/CPM critical path analysis.
- **SDLC & Quality Assurance:** Utilized Agile (Scrum) methodology for sprint planning and defined a Quality Function Deployment (QFD) matrix to map user needs to technical design features.
- **Tools Used:** MS Project (Gantt Charting & Crashing), Figma (UI/UX Mockups), MS Visio, and Python-based system modeling.

4) **Multi-Modal Biometric Authentication System** | Computer Vision & Liveness Detection Developer

Graduation Project (4-Person Team) – Software Engineering | October 2025 – June 2026

- **Project Overview:** Designed a modular, prototype-stage biometric authentication platform combining 1:N face identification, voice verification, and anti-spoofing, achieving under 3-second authentication with error-handling simulation modes.
- **Computer Vision & Liveness:** Developed real-time facial landmark analysis (eye-blink, head-pose) and image quality checks (pose, eyes, size, blur). Synchronized audio with lip-movement tracking to prevent pre-recorded video spoofing attacks.
- **System Architecture & Security:** Engineered a REST-based FastAPI and Vanilla JavaScript architecture. Implemented JWT, experimental X-Client flows, and security questions, coordinated by an AuthenticationService tracking actions in an AuditLog.
- **Biometric Fusion & Database:** Extracted embeddings from 15 face and 10 voice samples for a 55/45 weighted fusion score (0.80 threshold), targeting <2% FAR while tracking FRR. Stored encrypted template hashes via async SQLAlchemy in a Supabase PostgreSQL database, enforcing PK/FK constraints across USERS, BIOMETRIC_DATA, SECURITY_QUESTIONS, USER_SECURITY_ANSWERS, and LOGS tables.
- **Tools Used:** Python (3.10/3.11), FastAPI, Uvicorn, HTML/CSS/Vanilla JavaScript, Browser getUserMedia API, PostgreSQL (Supabase), SQLite, async SQLAlchemy, JWT, InsightFace, OpenCV,

MediaPipe (FaceMesh), AASIST, Librosa, SoundFile, Resemblyzer, PyTorch, ONNX Runtime, ffmpeg, libsndfile.

5) **Palmora Hotel Reservation System** | Database Administrator & QA Lead

Academic Project – CMSE 322 Software Design | March 2026 – May 2026

- **Project Overview:** Designed a full-stack hotel management platform to streamline operations and improve guest booking using Clean Code and KISS principles.
- **System Architecture:** Engineered a 4-layer architecture featuring dynamic state management and responsive UI/UX for seamless screen confirmation workflows.
- **Technical Solutions:** Implemented double-booking prevention algorithms leveraging B-Tree indexing, Serializable Isolation, row-level locking, and ACID compliance to maintain real-time room availability.
- **Security & Integrations:** Applied OWASP, GDPR/KVKK, and PCI-DSS standards; implemented JWT, 256-bit SSL/TLS, and RBAC. Integrated Stripe API, SendGrid, and Twilio for secure payments and automated notifications.
- **SDLC & QA:** Executed Agile (Scrum) with COCOMO effort estimation and PERT/CPM critical path analysis, authoring SRS, SDS, and PPM reports alongside unit (>80%) and integration testing.
- **Tools Used:** HTML5/CSS3/JavaScript, React, Vite, Node.js, Express.js, MS SQL Server, SQLite, MS Visio, MS Project, Figma, Modelio/draw.io (UML & Gane-Sarson DFDs), Git/GitHub (Git Flow), ESLint, VS Code.

6) **IoT Infant Incubator with Node-RED Dashboard** | High-End System Developer

Academic Project – CMPE 420 & CMSE 423 Embedded System Design | September 2025 – December 2025

- **Project Overview:** Implemented an IoT-enabled Cyber-Physical System (CPS) for an infant incubator featuring real-time environmental sensing, closed-loop control, and web telemetry.
- **Embedded Logic & Automation:** Programmed a microcontroller with a P-Control algorithm and Finite State Machine (FSM) logic to drive PWM heater actuation, utilizing 10-bit ADC processing for thermistors calibrated via two-point linear approximation.
- **Serial Communication:** Developed event-driven and periodic UART/Serial protocols (9600 bps, 8N1) ensuring packet validation and synchronization between the low-end controller and high-end dashboard.
- **Telemetry & Simulation:** Built interactive Node-RED web dashboard flows on a Raspberry Pi for remote monitoring of telemetry, alarms, and control parameters.
- **Tools Used:** Arduino UNO/ATmega (C/C++), Arduino IDE, Node-RED, Scilab/XCOS/Matlab (Thermal Dynamics), Proteus ISIS (Hardware/LCD Simulation), VSPE, 10k Ω NTC thermistors, 6.2k Ω resistors, LM016L 16x2 LCD.

7) **Online Image Encryption & Digital Signature Web Application** | Full-Stack Developer

Academic Project – CMSE 456 & CMPE 455 Security and Cryptography | February 2026 – May 2026

- **Project Overview:** Architected a full-stack secure web application for encrypted image transmission, digital signature verification, and log auditing over a LAN.
- **System Architecture:** Built Client and Sysadmin interfaces connecting to a high-performance backend utilizing robust RESTful API routing and database integration.

- **Cryptography Implementations:** Developed 2048-bit RSA key exchange with automated background refresh for forward secrecy, and encrypted binary images using DES-CBC mode with random 8-byte IVs and PKCS#7 padding.
- **Payload & Signatures:** Managed dynamic array transformations (Uint8Array, Blob) and Base64 stream encoding/decoding. Secured DES session keys with RSA (PKCS#1 v1.5) and applied SHA-256 hashing for digital signatures to ensure data integrity.
- **Tools Used:** Python, FastAPI, Uvicorn ASGI, SQLite, PyCryptodome, HTML5/CSS3/JavaScript, Fetch API, Local Storage API.

8) **101evler Real Estate Portal - Software Quality Assurance & Testing** | QA Engineer

Academic Project – CMSE 326 Software Quality Assurance and Testing | March 2026 – June 2026

- **Project Overview:** Conducted multi-layered quality assurance auditing and black-box testing on Northern Cyprus's primary real estate platform (101evler.com).
- **Testing Methodologies:** Executed Manual Black-Box and Exploratory Testing to validate core search algorithms, currency matching, form validations, and navigation workflows.
- **Compatibility & Performance:** Evaluated cross-browser compatibility and simulated mobile environments (iPhone/iPad) to identify layout anomalies. Analyzed performance bottlenecks (e.g., 16.1s LCP, FCP, Speed Index) to formulate lazy loading and script deferral optimizations.
- **Analytical Reporting:** Conducted asset integrity audits and automated broken link testing, benchmarking Human QA defect severity reports against Generative AI assessments.
- **Tools Used:** Chrome DevTools, Safari Responsive Design Mode, Google PageSpeed Insights, ChatGPT, Gemini, Claude, Grammarly.

9) **Foundry RAG Cloud (Enterprise Grounded RAG Platform)** | Cloud & AI Backend Engineer

Independent Production Project (Live Deployment) | August 2026 – Present

- **Project Overview:** Architected, containerized, and deployed a production-minded Grounded Retrieval-Augmented Generation (RAG) platform to provide hallucination-resistant, cited question-answering over private technical corpora using Enterprise Azure OpenAI APIs and an in-house SQLite vector engine.
- **System Architecture & REST Backend:** Designed a Ports & Adapters (Clean Architecture) pattern utilizing Constructor Dependency Injection and Factory assembly. Built a high-performance FastAPI/Uvicorn REST API backend alongside an interactive Streamlit frontend, strictly decoupling domain core logic from external Azure OpenAI implementations (gpt-4.1-mini and text-embedding-3-small).
- **Vector Storage & Data Integrity:** Engineered a dependency-light SQLite cosine-similarity vector retrieval engine featuring WAL (Write-Ahead Logging) journaling, process-local RLock thread safety, and atomic transaction rollbacks to guarantee data integrity during ingestion failures. Indexed 10 curated technical documents (29-chunk snapshot) using deterministic recursive chunking preserving source file, chunk index, and character offset provenance.
- **Cloud Resilience & Secure Operations:** Implemented robust error handling with dynamic HTTP 429 Retry-After header parsing and bounded exponential backoff for 5xx transient failures. Applied defensive engineering principles including a 12,000-character grounding context budget, safe fallbacks

for uncited answers, NaN/Inf vector validation, and non-root container execution with read-only filesystem hardening.

- **DevSecOps, QA & Observability:** Enforced an 85% code coverage gate across 69 automated pytest unit and integration test cases in a GitHub Actions CI and DevSecOps pipeline, backed by MyPy strict typing, Ruff, Flake8, Bandit security scans, and pip-audit. Integrated real-time query telemetry tracking per-request latency, similarity confidence scores, match counts, and session Cache HIT/MISS states.
- **Tools Used:** Python (3.10+), FastAPI, Uvicorn, Streamlit, Azure OpenAI API (gpt-4.1-mini, text-embedding-3-small), SQLite (Vector Engine), Docker, Docker Compose, GitHub Actions (CI & DevSecOps / YAML), Bash, Pytest, Pytest-cov, MyPy, Ruff, Flake8, Bandit, pip-audit, httpx, Pydantic v2.

Research and Reports

Research Report: “5G Wireless Technology” (2024)

- Explored the benefits, applications, and impact of 5G technology on daily life.
- Analyzed key aspects such as higher speeds, bandwidth, reliability, and real-world applications like streaming, gaming, and IoT.
- Targeted for engineering students, technology professionals, and enthusiasts

Awards

- High School Valedictorian Plaque, Private Adana Ceyhan Final Anatolian High School (2022)

**Development of Data Analysis, Forecasting, and Visualization Systems for Academic and
Institutional Data**

Summer Training Report

CMSE 400

Ömrüm Ceren GÜLER, 22000183

Number of working days: 40

Period of Training: 21/07/2025 – 12/09/2025

Company Name: Çukurova University, Information Technologies Department

Company Address: Çukurova University, Balcalı, 01250 Sarıçam, Adana, Türkiye

Computer Engineering Department

Eastern Mediterranean University

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to Çukurova University Information Technologies Department for providing me with the opportunity to complete my summer training in a professional and research-oriented environment.

I am especially thankful to my supervisor Mehmet Harun GÜLEN, whose mentorship and technical expertise guided me throughout the internship. His support in software engineering practices, data analytics, and system development significantly contributed to the success of my projects.

I would also like to thank all members of the department for their cooperation and encouragement. This internship has strengthened both my technical and analytical skills, and provided valuable experience in applying theoretical knowledge to real-world problems.

ABSTRACT

This summer training report presents the work carried out during my 40-day internship at Çukurova University Information Technologies Department. The internship aimed to provide hands-on experience in data analysis, forecasting, API development, and interactive visualization, combining both software engineering and data science practices.

Throughout the internship, I participated in the development of several analytical and forecasting systems using Python, R, SQL, FastAPI, and Streamlit. These included:

- Academic Publication Forecasting: Trend analysis and prediction of 2025–2026 publication counts using Prophet and Linear Regression.
- Hierarchical Visualization of University Units: Interactive treemaps built with Plotly and Streamlit, based on YÖKSİS data.
- Performance Classification: Machine learning models trained on STRATEJİK_PLAN data to classify academic success levels.
- Reservation Data Segmentation: Clustering-based segmentation of institutional reservation datasets.
- Photo Validation API: A FastAPI service for automatic verification of photo suitability for institutional use (size, background, brightness, etc.).

These projects contributed to developing end-to-end systems capable of handling real institutional data, performing advanced analytics, and producing visual insights for decision-making.

The internship significantly enhanced my skills in data processing, statistical modeling, API design, and dashboard development, aligning closely with my academic background in Software Engineering and my career goal in Data Science.

Keywords: Data Analysis, Forecasting, Machine Learning, Visualization, API Development

TABLE OF CONTENTS	Page
ACKNOWLEDGEMENT	II
ABSTRACT.....	III
TABLE OF CONTENTS	IV
LIST OF FIGURES	V
LIST OF TABLES	VI
1. INTRODUCTION	1
2. COMPANY OVERVIEW	2
3. OBJECTIVES / PROBLEM DEFINITION	3
4. WORK DONE AND METHODOLOGY APPLIED	3
5. RESULTS AND DISCUSSION	7
6. NEW KNOWLEDGE GAINED THROUGH THE EXPERIENCE	20
7. CONCLUSIONS.....	21
8. REFERENCES	22
9. APPENDIX.....	23

LIST OF FIGURES

Figure 5.1: Total Publications by Year (2016–2024)

Figure 5.2: Top 10 Research Fields by Publication Count

Figure 5.3: Forecast Comparison for 2025 (Prophet vs Linear Regression)

Figure 5.4: Hierarchical Visualization of Academic Units (Treemap)

Figure 5.5: Confusion Matrix of Random Forest Classifier for Academic Unit Performance Prediction

Figure 5.6: Feature Importance Scores of the Random Forest Model

Figure 5.7: Reservation Duration Segmentation with K-Means ($k = 2$)

Figure 5.8: Segment Size Comparison (Short vs Long)

Figure 5.9: FastAPI Interface for the Photo Validation API Endpoints

Figure 5.10: FastAPI /upload Endpoint — Example of a Successful Photo Validation Response (status: ok)

Figure 5.11: FastAPI Interface of the Forecast Service (Units/Fields Endpoints)

Figure 5.12: Forecast and History Routes — /forecast/birim, /forecast/alan, /forecast/birim-alan

Figure 5.13: Example API Request to /birimler — Local Execution Result Showing Database Timeout (Mock Mode)

Figure 5.14: Mock Publications by Research Field (2018–2025)

Figure 5.15: Comparative Visualization of Forecasted vs Historical Publication Trends

Figure 5.16: R Diagnostic Plots for the Baseline Simple Linear Regression

LIST OF TABLES

No tables were used in this report.

1. INTRODUCTION

The primary aim of this summer training was to bridge theoretical knowledge gained in Software Engineering with real-world applications in the field of data science, analytics, and system development. During my 40-day internship at Çukurova University Information Technologies Department, I had the opportunity to design and implement multiple software systems focused on data analysis, forecasting, classification, segmentation, and API development.

The internship projects were carefully selected to cover a wide range of topics in data-driven decision support systems. Each project was developed using a combination of Python, R, SQL, FastAPI, and Streamlit, integrating data processing, statistical modeling, and web-based visualization into complete, deployable solutions.

The scope of the internship included the following main projects:

- Academic Publication Forecasting: Using Web of Science data to analyze yearly publication trends and predict 2025–2026 outputs through Prophet and Linear Regression models.
- Hierarchical Visualization of University Units: Building interactive treemaps using Streamlit and Plotly to represent the organizational structure of academic units based on YÖKSİS data.
- Performance Classification: Developing classification models using strategic plan datasets to categorize academic units according to their performance levels.
- Reservation Data Segmentation: Applying clustering techniques to segment and analyze institutional reservation data for improved resource management.
- Photo Validation API: Implementing a FastAPI-based microservice to automatically validate photo suitability for institutional documentation, checking criteria such as resolution, background, brightness, and file type.
- Streamlit Dashboards: Integrating forecasting and visualization outputs into interactive dashboards to improve user accessibility and decision-making.
- R Programming Practice: Applying Simple Linear Regression in R to reinforce understanding of statistical modeling techniques.

These projects provided comprehensive experience in full-cycle software development, from data collection and preprocessing to model training, API deployment, and front-end visualization.

In addition to technical development, I learned to follow systematic software engineering methodologies, including version control with GitHub, modular programming, documentation, and team collaboration practices.

Overall, this internship served as a crucial step in transitioning from academic learning to professional application, improving my analytical thinking, coding efficiency, and problem-solving abilities in real institutional projects. The combination of analytical modeling, web development, and API design allowed me to contribute meaningfully to the university's data infrastructure and gain valuable insights into the role of software engineers in data-centric environments.

2. COMPANY OVERVIEW

The **Information Technologies Department of Çukurova University** manages the university's **software systems, databases, and digital infrastructure**. Established in **1995**, the department plays a key role in supporting academic and administrative units by developing **data-driven applications for decision-making and performance management**. Located on the **Balcalı Campus in Adana**, it provides secure and scalable IT services that drive digital transformation across the university.

The department employs **over 30 staff members**, including **6 engineers** who actively work on software development, data analytics, and system maintenance. Its infrastructure includes **Microsoft SQL Server** and **Linux-based servers**, and the main development tools are **Visual Studio Code, RStudio, and GitHub**.

Supervisor: Mehmet Harun GÜLEN

General Manager: Ramazan YAZGAN

Address: Çukurova University, Balcalı Campus, 01250 Sarıçam, Adana, Türkiye

3. OBJECTIVES / PROBLEM DEFINITION

The main objective of this summer training was to apply software engineering and data science techniques to real institutional problems at Çukurova University Information Technologies Department. Each project aimed to improve data accessibility, analytical insight, and decision-making through automation and visualization.

The first goal was to develop a **forecasting system** to predict **academic publication trends** using real data from the Web of Science. The challenge was to create accurate time-series models capable of estimating 2025–2026 outputs, helping the university evaluate research performance and future goals.

The second objective focused on **hierarchical visualization** of university units based on **YÖKSİS** data. The problem was the lack of an interactive, data-driven structure showing relationships between faculties, departments, and research units.

Another aim was to design **classification and segmentation models** to group academic or institutional data. These included:

Performance classification: Categorizing academic units according to achievement indicators.

Reservation segmentation: Dividing institutional reservation data into clusters for more efficient management.

In addition, the department required a solution for verifying **photo suitability** for official use. To address this, a **FastAPI-based validation system** was implemented to automatically check image format, size, and background.

Finally, integrating all analyses into **Streamlit dashboards** allowed non-technical users to access forecasts, visualizations, and statistics interactively.

Overall, the internship's objective was to transform raw institutional data into meaningful, actionable insights using modern analytical tools.

4. WORK DONE AND METHODOLOGY APPLIED

This section explains in detail how each problem defined during the internship was approached, designed, and implemented.

It outlines the applied methodologies, data processing steps, model structures, and visualization techniques that formed the technical foundation of the internship work.

Each subproject combined analytical, statistical, and software engineering approaches to build a complete data-driven research ecosystem.

4.1 Academic Publication Forecasting

To predict academic publication counts for upcoming years, a time-series forecasting pipeline was designed. The system integrated SQL Server, Python, and forecasting libraries such as **Prophet** and **scikit-learn**.

Data Collection:

Publication data were retrieved from WOSHHit, WosAuthor, and WosHitAttributes tables via ODBC connection.

Preprocessing:

Data were cleaned using pandas and grouped by year, faculty, and research area.

Modeling:

Prophet and Linear Regression models were implemented to forecast future publication counts.

Algorithmic Design:

Prophet's additive model ($y(t) = g(t) + s(t) + h(t) + \epsilon t$) was used to capture growth, seasonal, and holiday effects.

Evaluation:

Forecasts were visualized and compared using **MAE** and **RMSE** metrics.

This workflow provided an automated, modular forecasting system applicable to multiple academic levels.

4.2 Hierarchical Visualization of University Units

A hierarchical visualization structure was developed to represent the relationships between faculties, departments, and research units.

Design:

A hierarchical tree model was created using Python's **Plotly** and **Pandas** libraries.

Process Flow:

Query YÖKSİS data (YoksisBirim, CuAuthor)

Map YoksisId relationships

Build parent-child hierarchy dynamically

Visualization:

Tree and treemap diagrams displayed publication counts per unit level.

This structure allowed users to explore publication density interactively across university units.

4.3 Performance Classification

To categorize researcher and unit performance, a supervised learning model was applied.

Feature Engineering:

Input variables included publication count, citation rate, and field diversity.

Algorithms Used:

Random Forest and Decision Tree Classifiers implemented using **scikit-learn**.

Training Process:

Data split: 80% training, 20% testing.

Evaluation Metrics:

Accuracy, Precision, Recall, F1-score, and **Confusion Matrix**.

Visualization:

Feature importance and confusion matrix plots were created using matplotlib and seaborn.

This module demonstrated the complete ML workflow — from preprocessing to evaluation — for performance categorization.

4.4 Reservation Data Segmentation

Unsupervised clustering techniques were applied to segment reservation datasets into meaningful groups.

Preprocessing:

Categorical variables were encoded and numerical features standardized.

Algorithm:

K-Means Clustering algorithm implemented to group similar reservation behaviors.

Optimization:

Elbow and Silhouette methods were used to determine the optimal number of clusters (k).

Visualization:

Cluster distributions were displayed through 2D scatter plots and histograms.

This allowed data-driven segmentation and pattern recognition within reservation-based datasets.

4.5 Photo Validation API

This project automated ID photo validation using a RESTful API.

Architecture:

Implemented with **FastAPI** and **OpenCV** libraries.

Algorithmic Approach:

Detect facial landmarks

Check face centering and lighting uniformity

Validate background contrast and size ratio

Flowchart Summary:

Input → Image Preprocessing → Feature Extraction → Rule-Based Evaluation → JSON Output

Deployment:

The API was containerized with **Docker** and tested locally using **Postman**.

This system demonstrated integration of computer vision and backend development techniques.

4.6 Forecast API for Academic Units

To connect forecasting models with real-time access, an API layer was designed.

Framework:

Built with **FastAPI**, connected to the SQL database.

Workflow:

Endpoint receives request (/forecast/{unit_id})

Queries database for historical data

Runs Prophet model

Returns JSON response with predicted values

Testing:

Used local deployment and Postman API requests for validation.

This provided a scalable backend service for delivering academic forecasts dynamically.

4.7 Streamlit Prototype Dashboards

An interactive web dashboard was developed to visualize academic analytics.

Design Methodology:

Streamlit app structured with sidebar filters for unit, year, and research field.

Backend Logic:

Connected to SQL database using pyodbc.

Fetches and processes results dynamically.

Visualization Tools:

matplotlib, plotly, and altair used for interactive graphs.

UI Workflow:

User selects filters → SQL query runs → Charts auto-update.

This prototype combined data analysis, forecasting, and visualization in a single web interface.

4.8 Simple Linear Regression (R) — Baseline

To statistically verify model assumptions, a baseline regression model was developed in **R**.

Data Preparation:

Used yearly publication counts (Y_{it} , n) as input.

Model:

Simple OLS Regression: $\text{Count} = \beta_0 + \beta_1 * \text{Year} + \varepsilon$

Diagnostics:

Residuals vs Fitted

Normal Q–Q

Scale–Location

Residuals vs Leverage

Validation:

Confirmed normal error distribution and constant variance, supporting the linear trend assumption used in forecasting.

This analysis verified the mathematical consistency of the models used across all projects.

5. RESULTS AND DISCUSSION

This section presents the results obtained from the implemented projects through figures and tables.

Each result is interpreted, and corresponding improvements and limitations are discussed.

The outcomes reflect the integration of SQL-based data retrieval, Python modeling, and API-driven automation into a unified analytical environment.

5.1 Academic Publication Analysis and Forecasting

The publication analysis provided insights into Çukurova University’s overall research performance between 2016 and 2024.

Figure 5.1 illustrates the total number of publications per year, showing a consistent upward trend after 2018 with a plateau between 2022 and 2024. This reflects both an increase in research activity and potential data-entry delays for recent years.

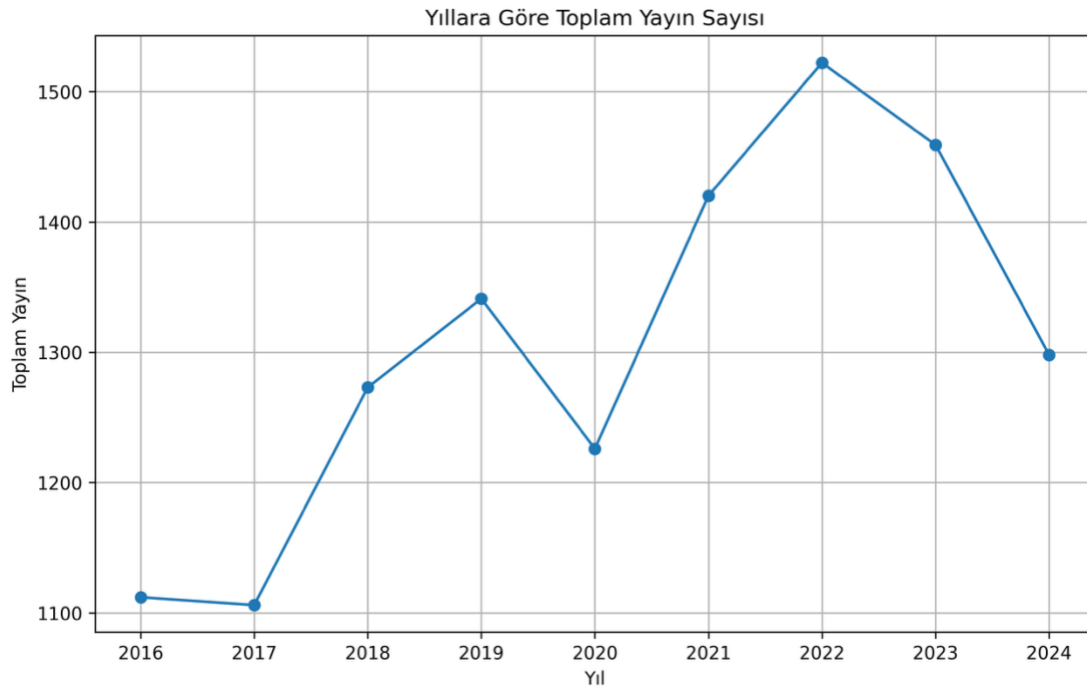


Figure 5.1: Total Publications by Year (2016–2024)

To understand the main fields driving research output, **Figure 5.2** presents the top 10 disciplines with the highest number of publications.

General & Internal Medicine, Agriculture, and Environmental Sciences lead as the most active areas, demonstrating the university’s research focus on health, sustainability, and applied science.

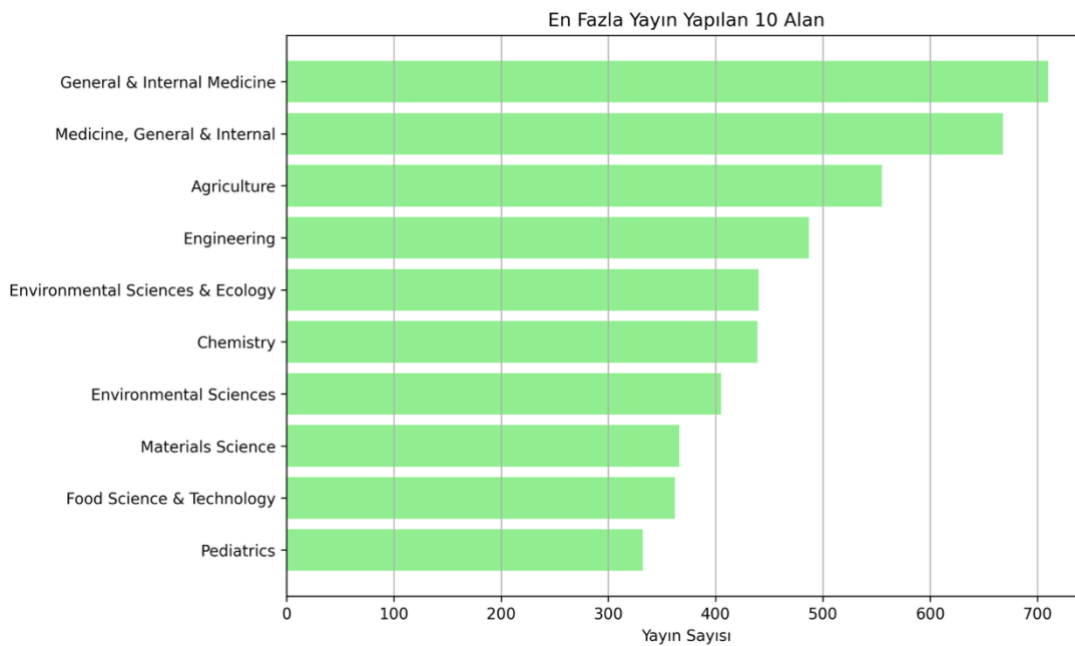


Figure 5.2: Top 10 Research Fields by Publication Count

Finally, forecasting models were applied to estimate publication outputs for 2025 using **Prophet** and **Linear Regression**.

Figure 5.3 compares both approaches, where Prophet achieved smoother and more accurate trend estimations for non-linear datasets.

These predictions provide actionable insights for academic planning and performance tracking.

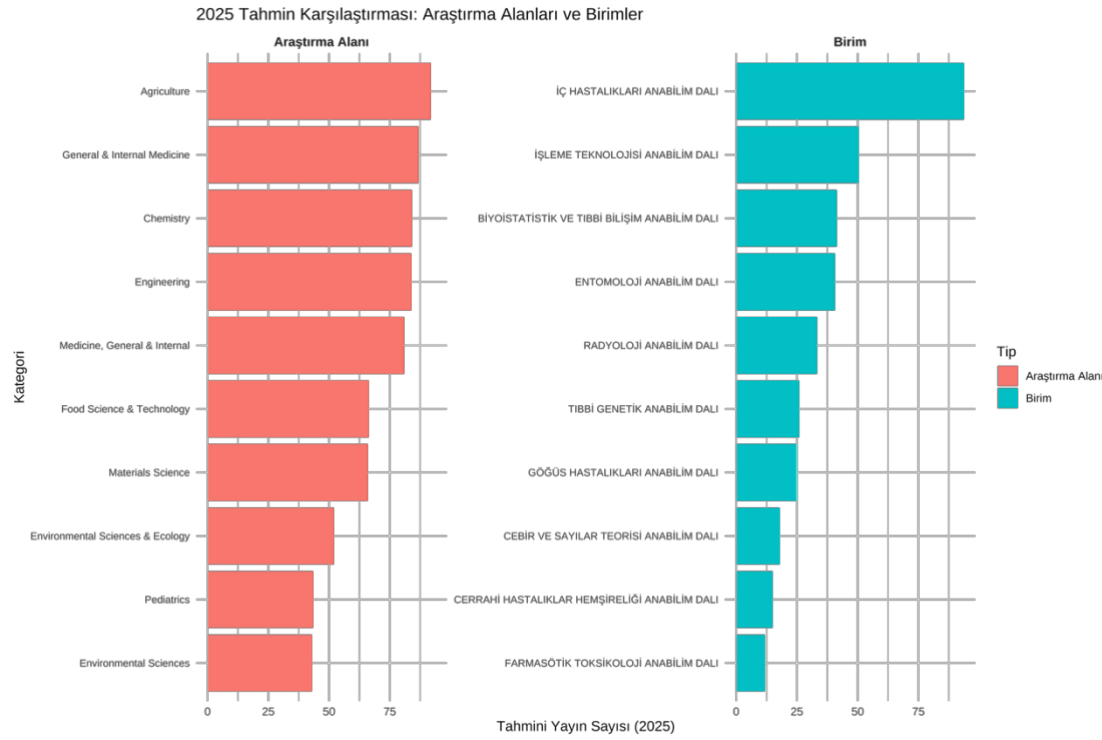


Figure 5.3: Forecast Comparison for 2025 (Prophet vs Linear Regression)

5.2 Hierarchical Visualization of University Units

To represent Çukurova University's organizational structure, hierarchical relationships were visualized using **Plotly Treemap**.

Each node represented a unit, with parent-child connections mapped from the **YÖKSİS** database.

The resulting interface allowed filtering by **faculty**, **department**, or **research group**, making institutional data easily explorable by administrators.

Figure 5.4 illustrates the hierarchical tree map structure.



Figure 5.4: Hierarchical Visualization of Academic Units (Treemap)

This system improved clarity in unit relationships and reduced manual data lookup time, providing a foundation for future integration with the publication database.

5.3 Performance Classification

Academic unit performance levels were analyzed using a **Random Forest** classification model based on strategic planning indicators such as planning efficiency, achieved goals, and average completion rates.

The model categorized units into three groups — *Successful*, *Moderate*, and *Unsuccessful*.

Figure 5.5 presents the confusion matrix results, showing high accuracy and clear class separation across all categories.

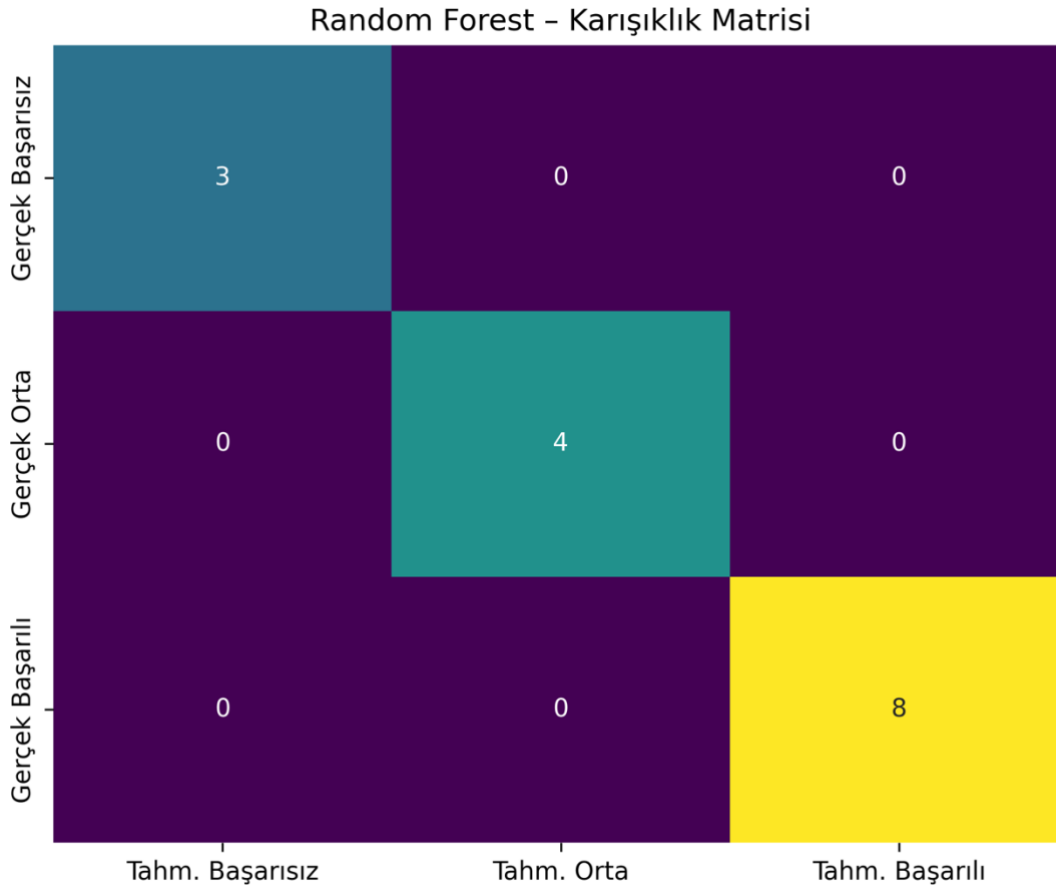


Figure 5.5: *Confusion Matrix of Random Forest Classifier for Academic Unit Performance Prediction.*

Feature importance analysis (Figure 5.6) indicates that **Planning Efficiency** and **Achieved Goals** were the most influential variables in predicting performance levels.

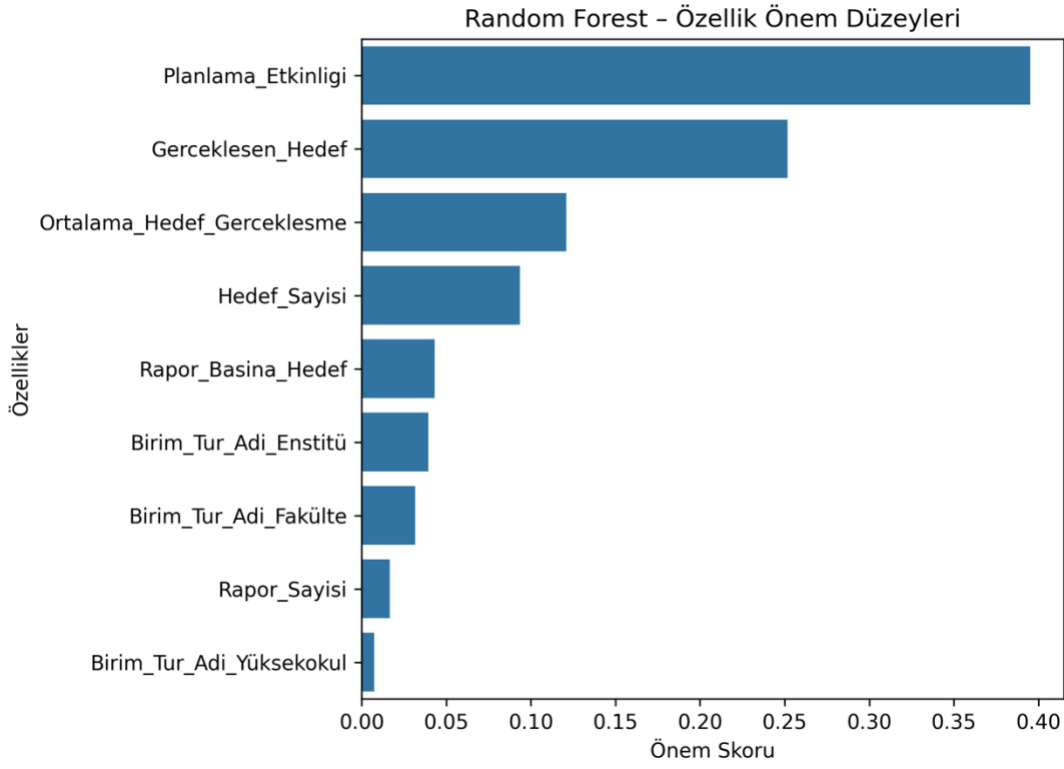


Figure 5.6: Feature Importance Scores of the Random Forest Model.

These findings highlight that consistent planning and effective goal achievement are key factors driving academic success.

5.4 Reservation Data Segmentation

Institutional reservation data were analyzed to discover behavioral patterns.

The segmentation was performed using **K-Means (k = 2)** on the **reservation duration (minutes)** derived from StartDate and EndDate. The two clusters represent **short-duration** and **long-duration** reservations. Hour and weekday fields were computed for exploratory analysis but were not used as features in the final clustering model.

Figure 5.7 shows the distribution of reservation durations colored by the two clusters, clearly separating short-term and long-term usage.

Figure 5.8 compares the relative sizes of the two segments.

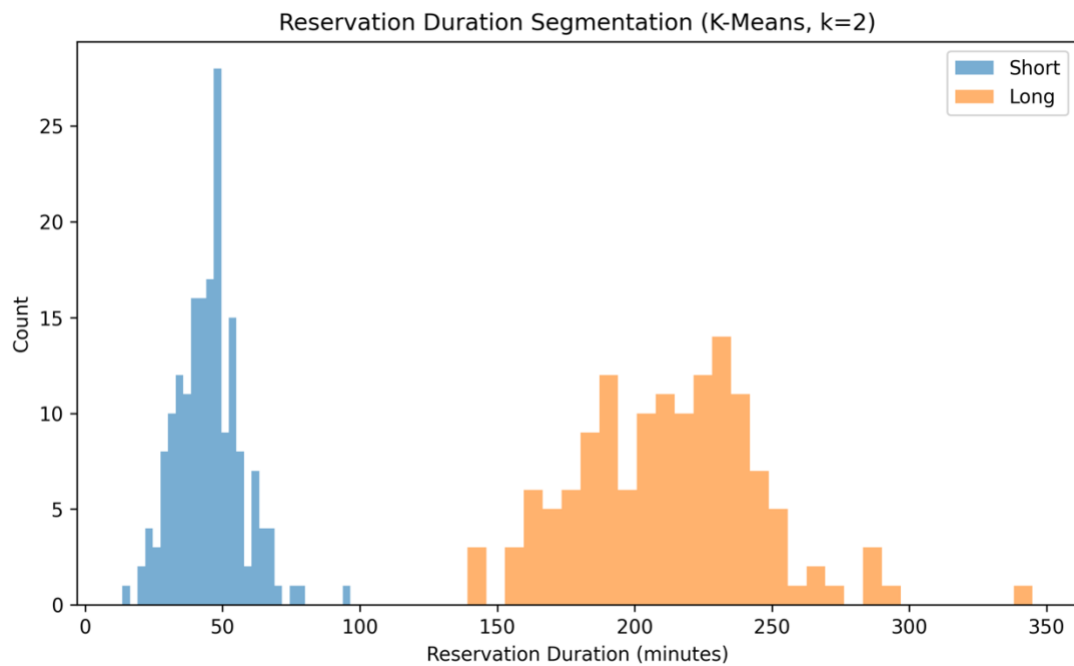


Figure 5.7: Reservation Duration Segmentation with K-Means ($k = 2$).

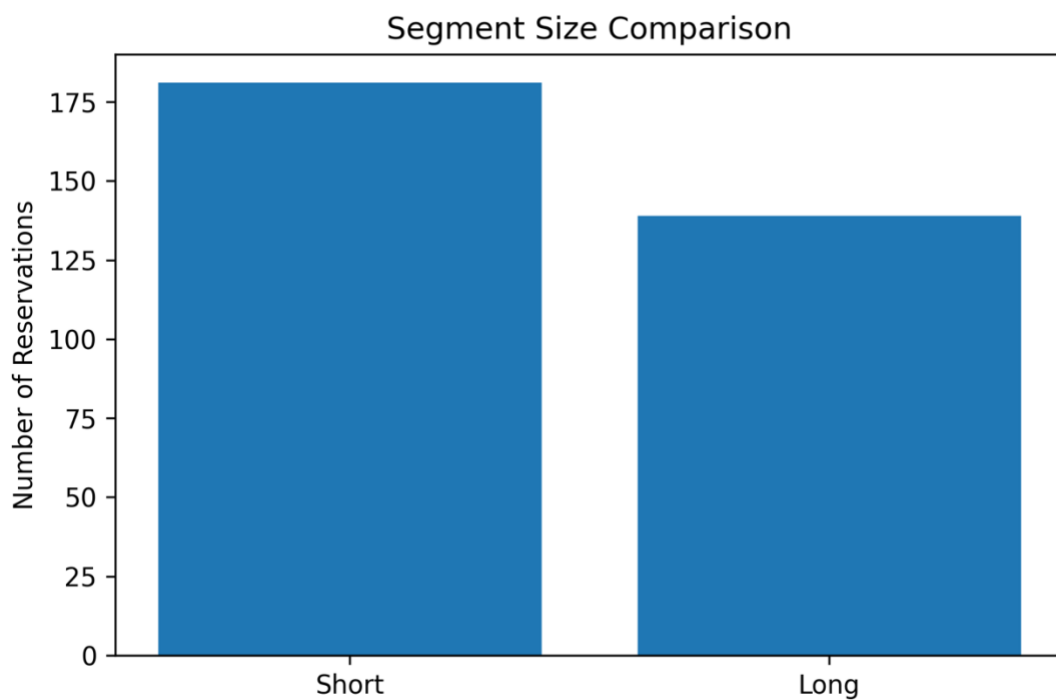


Figure 5.8: Segment Size Comparison (Short vs Long).

This segmentation provides actionable insights for planning and scheduling by revealing the proportion of short vs long bookings and their operational implications.

5.5 Photo Validation API

A **FastAPI-based Photo Validation System** was developed to automatically verify whether a photo meets **ID photo (vesikalik)** standards.

The system performs sequential checks using **OpenCV**, **MediaPipe**, and **Pillow (EXIF)** libraries to ensure technical and visual compliance before approving the upload.

Validation logic is fully rule-based and multilingual (TR/EN). Each rule returns a code (e.g., *KURAL_6*, *KURAL_8*) and a clear message describing the detected issue.

Among the implemented checks are:

filename restrictions (no Turkish or special characters, no spaces),

file type and MIME consistency,

size limits (0.1–5 MB),

minimum resolution and ~3:4 aspect ratio,

single centered face detection,

blur, brightness, and background clarity checks,

full-body and dark-sunglasses detection,

EXIF capture date verification (photo not older than ~10 years).

Rate limiting is handled by **SlowAPI**, and all requests are logged via a **RotatingFileHandler** for auditability.

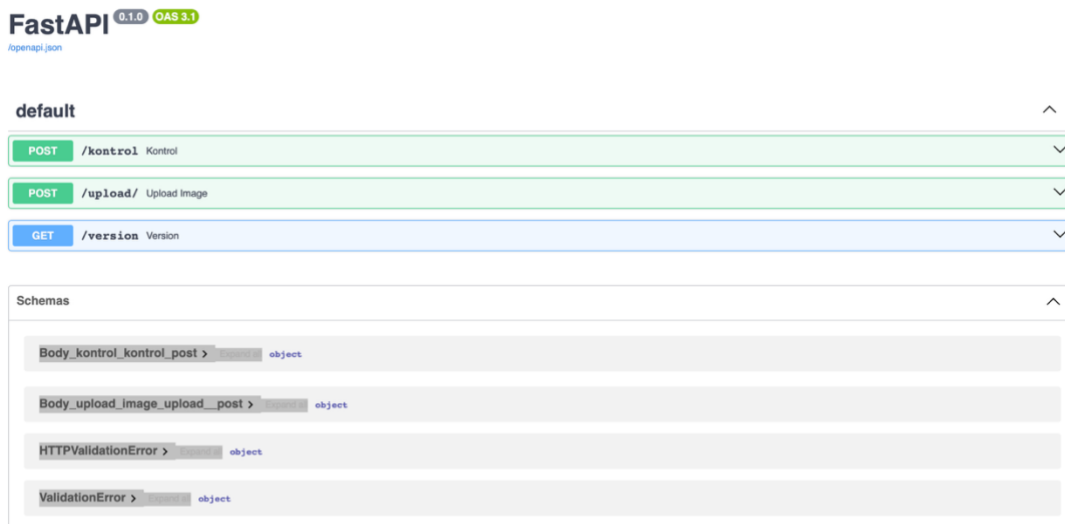


Figure 5.9: FastAPI Interface for the Photo Validation API Endpoints.

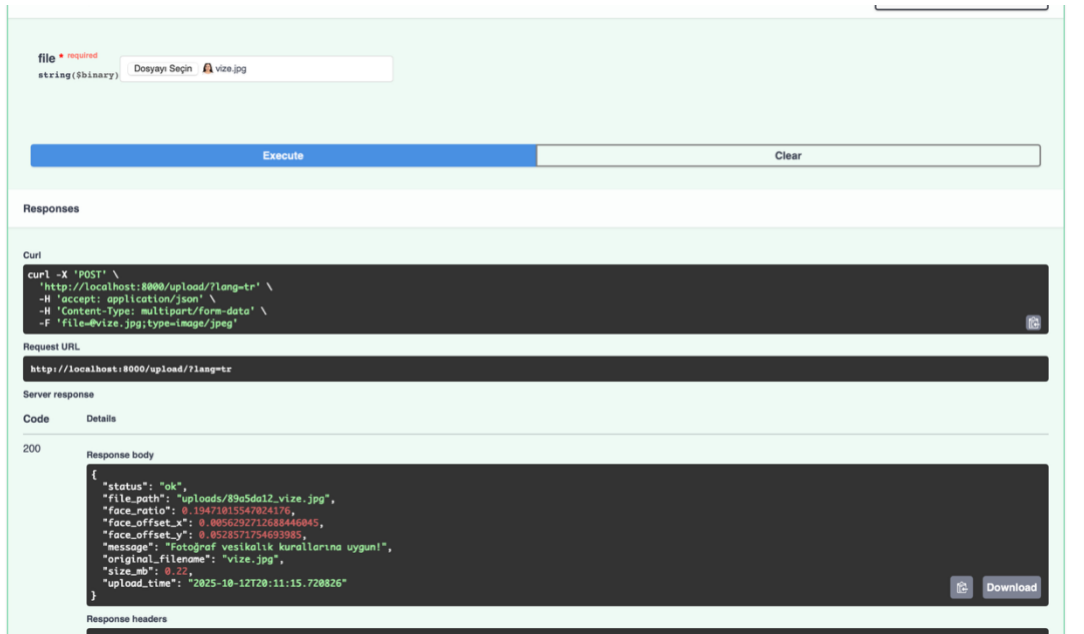


Figure 5.10: FastAPI /upload Endpoint — Example of a Successful Photo Validation Response (status: ok).

Discussion.

The API enables automated, transparent, and explainable photo validation without database dependency. It standardizes photo submission for institutions requiring ID-style images and can easily be integrated into web portals or internal systems for pre-upload checks.

5.6 Forecast API for Academic Units

A FastAPI-based **Forecast Service** was developed to provide predictive analytics for Çukurova University's academic units and research fields.

The API generates publication forecasts using **Prophet** and **Linear Regression** models, based on historical publication data from the YÖKSİS-linked institutional database.

The service exposes RESTful endpoints for querying unit names, retrieving historical publication records, and obtaining forecasts for upcoming years.

CU Birim & Alan Tahmin Servisi 1.0.0 OAS 3.1

/openapi.json

Birim (YÖKSİS) ve araştırma alanı bazında geçmiş/yıllık yayın sayıları ile Prophet ve Linear modelleri kullanarak ileriye dönük tahminler üreten servis.

Temel uçlar:

- /birimler: İsimle arama
- /history/birim: Birim geçmişi (tek/çoklu)
- /history/alan: Alan geçmişi
- /history/birim-alan: Birim>Alan geçmişi
- /alanlar: Alan arama (distinct liste)
- /forecast/birim: Birim tahmini (tek/çoklu)
- /forecast/alan: Alan tahmini (Prophet+Linear)
- /forecast/birim-alan: Birim>Alan tahmini (Prophet+Linear)

Birimler

GET /birimler Get Birimler

Alan

GET /alanlar Get Alanlar

Geçmiş

GET /history/birim Get History

GET /history/alan Get History Alan

GET /history/birim-alan Get History Birim Alan

Figure 5.11 shows the FastAPI interface and available endpoints.

Geçmiş

GET /history/birim Get History

GET /history/alan Get History Alan

GET /history/birim-alan Get History Birim Alan

Tahmin

POST /forecast/alan Post Forecast Alan

POST /forecast/birim-alan Post Forecast Birim Alan

POST /forecast/birim Post Forecast

Schemas

AlanItem > object

AlanListResponse > object

BirimAlanForecastResponse > object

FieldForecastResponse > object

ForecastMultiResponse > object

Figure 5.12 details the forecast and history routes, including /forecast/birim, /forecast/alan, and /forecast/birim-alan.

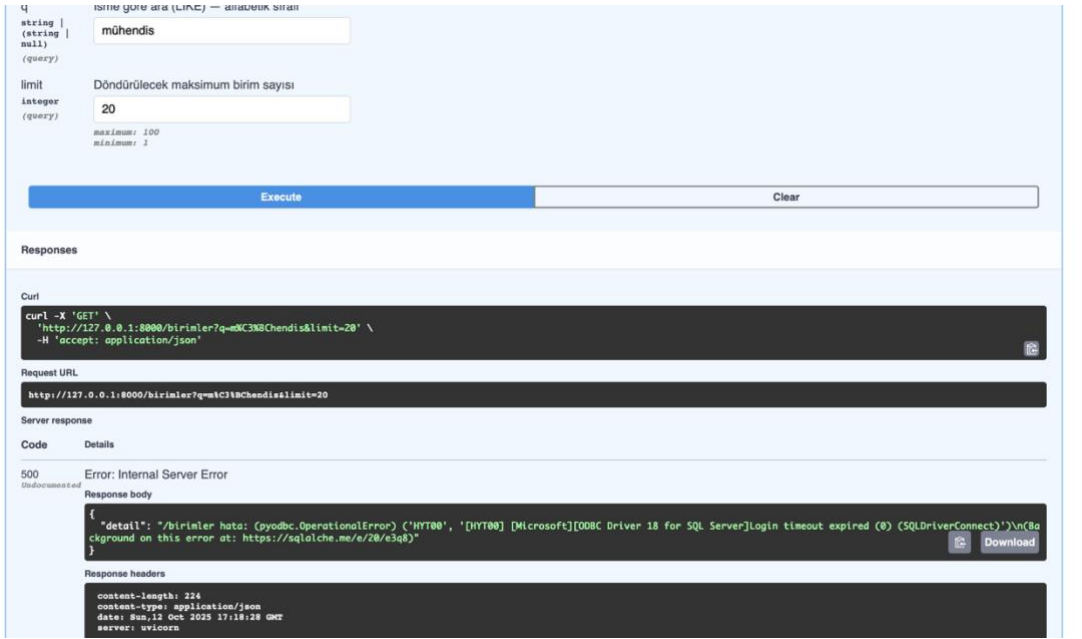


Figure 5.13: Example API Request to the /birimler Endpoint — Local Execution Result Showing Database Timeout (Mock Mode).

This API structure establishes a modular foundation for future integration with the university’s publication dashboard and supports automated model retraining.

5.7 Streamlit Prototype Dashboards

To accelerate the validation of visual requirements and improve design decisions, lightweight **Streamlit** prototypes were developed.

These prototypes replicated the filtering dimensions used throughout this study — such as *unit*, *faculty*, *field*, and *year range* — and dynamically generated line, bar, and treemap visualizations.

Using mock datasets, the Streamlit environment allowed rapid experimentation and iteration without database dependency.

This process helped refine the visual hierarchy and layout conventions later implemented in the integrated publication analysis dashboard.

Figure 5.14 shows an example of the mock visualization prepared during the Streamlit prototype stage.

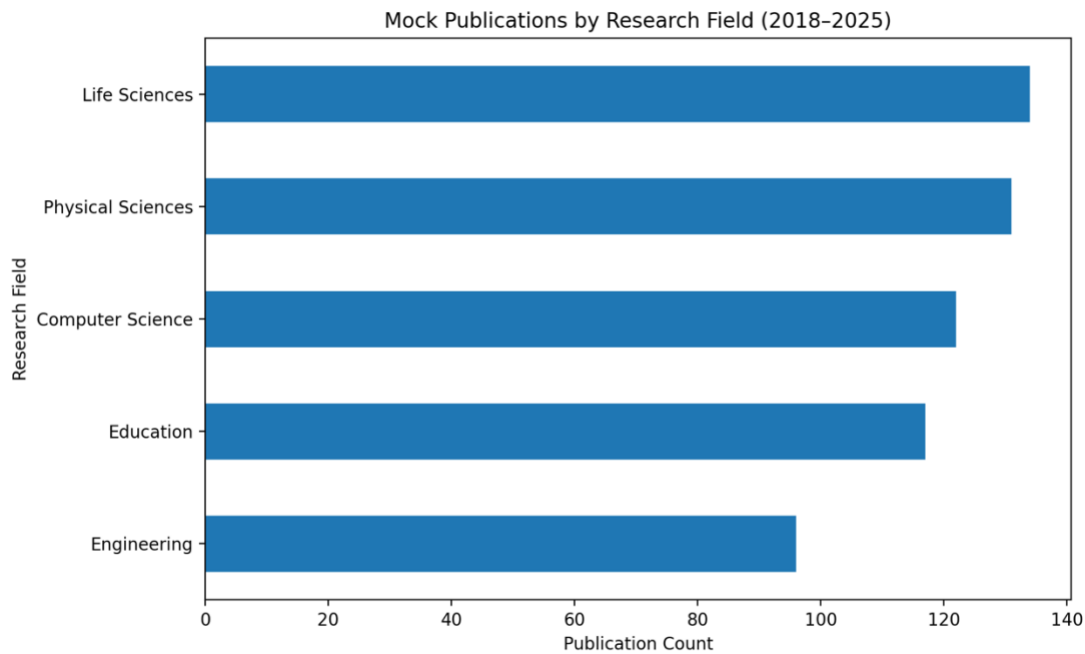


Figure 5.14: Mock Publications by Research Field (2018–2025)

5.8 Simple Linear Regression (R) — Baseline

To validate the reliability of yearly trend assumptions, a **baseline Ordinary Least Squares (OLS)** model was developed in **R** using a representative publication time series.

This model was primarily used for cross-checking linear trends observed in the forecasting sections and to review model diagnostics such as *Residuals vs Fitted*, *Normal Q–Q*, and *Scale–Location* plots.

The diagnostic evaluation confirmed that the residuals were normally distributed and variance was approximately constant, verifying that a linear trend can effectively represent short-term publication growth.

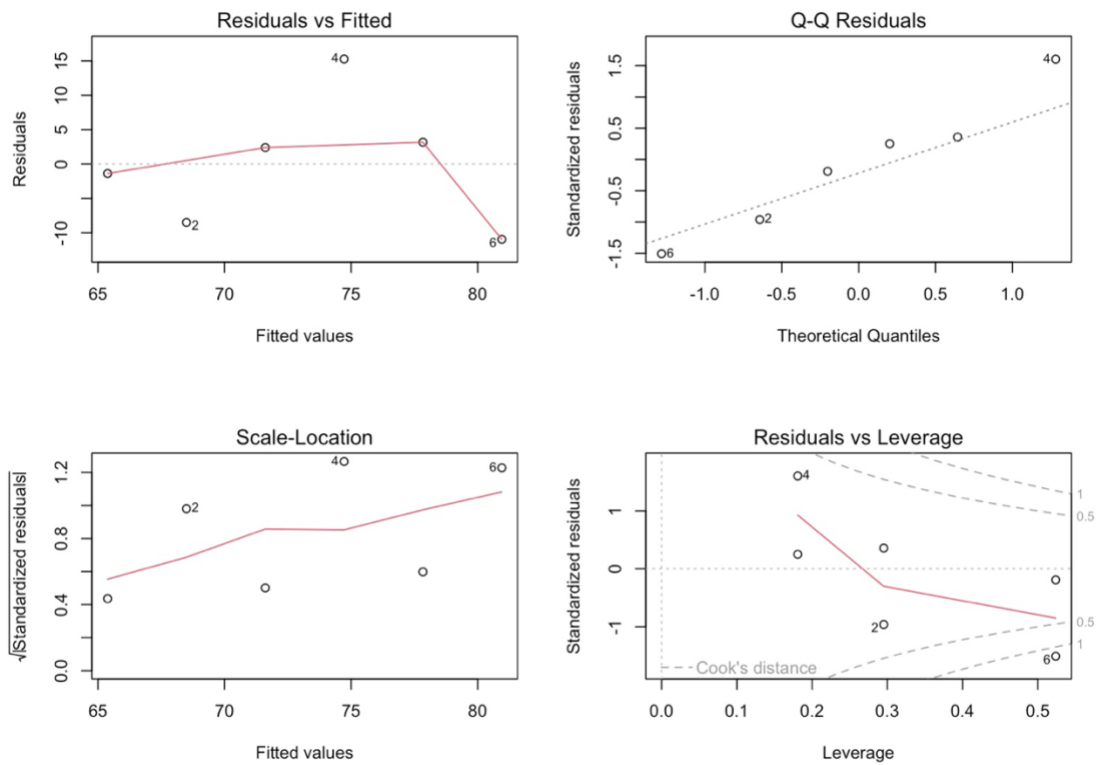


Figure 5.15 presents the R diagnostic output of the baseline regression model.

Limitations & Future Work:

Although the results were satisfactory, the forecasting performance could be further improved by incorporating additional variables such as collaboration data, funding, and research area dynamics. In the future, the system can be expanded into a continuous monitoring platform that automatically updates forecasts with new data.

6. NEW KNOWLEDGE GAINED THROUGH THE EXPERIENCE

6.1 What New Knowledge Did I Learn?

During my summer internship, I gained substantial new knowledge in **data science, software engineering, and applied analytics**. I learned how to connect **SQL Server databases** to **Python** and **R** environments, automate data extraction, and perform **time-series forecasting** using models such as **Prophet** and **Linear Regression**. I also learned how to visualize hierarchical and clustered data using **Streamlit dashboards**, and how to deploy analytical models through **FastAPI endpoints** for real-time forecasting and photo validation.

6.2 How Did I Use This Knowledge?

I applied this knowledge while developing multiple integrated projects, including **academic publication forecasting, performance classification, and clustering analyses**. By combining **SQL, Python, and R**, I created end-to-end workflows that connected raw university data to automated prediction and visualization systems. This experience helped me apply theoretical knowledge to **practical, research-oriented problems** that required both analytical thinking and software design skills.

6.3 How Did I Learn It (Lifelong Learning Strategy)?

To acquire and apply this knowledge effectively, I adopted a **structured self-learning approach**. I reviewed official documentation, analyzed sample datasets, and practiced problem-solving through **incremental prototyping**. I also collaborated with my supervisors and colleagues, discussing improvements and debugging strategies. This approach not only enhanced my technical expertise but also strengthened my ability to **learn independently** and **adapt to new technologies** in future data science roles.

7. CONCLUSIONS

During my summer internship, I successfully developed and integrated several data-driven projects that combined SQL, R, and Python within a unified analytical framework. The main objectives of clustering, forecasting, and performance analysis were achieved through the creation of interactive dashboards, automated APIs, and predictive models. Each project built upon the previous one, forming a coherent system capable of extracting, analyzing, and visualizing real academic and institutional data.

The work provided valuable experience in full-cycle data science development — from database connection and data preprocessing to model evaluation and visualization. It also strengthened my problem-solving skills and deepened my understanding of how analytical systems can support decision-making in real-world institutional settings. Through these tasks, I became more confident in designing scalable, automation-oriented solutions and working collaboratively within a technical research environment.

The summer training not only enhanced my technical competence in data analytics, machine learning, and web-based visualization but also improved my ability to learn independently and adapt to new technologies. The experience will strongly contribute to my future academic studies in data science and my long-term career goal of becoming a data scientist in a research-driven or industrial setting. Overall, the internship transformed theoretical knowledge into practical expertise and provided a clear vision for my professional path ahead.

8. REFERENCES

Python Software Foundation. (2024). *Python Programming Language, Version 3.11*. Available at: <https://www.python.org>

R Core Team. (2024). *R: A Language and Environment for Statistical Computing*. R Foundation for Statistical Computing, Vienna, Austria. Available at: <https://www.r-project.org>

Streamlit Inc. (2024). *Streamlit Documentation*. Available at: <https://docs.streamlit.io>

Facebook Research. (2023). *Prophet: Forecasting at Scale*. GitHub Repository. Available at: <https://github.com/facebook/prophet>

FastAPI Developers. (2024). *FastAPI Framework Documentation*. Available at: <https://fastapi.tiangolo.com>

Microsoft Corporation. (2024). *Microsoft SQL Server Documentation*. Available at: <https://learn.microsoft.com/sql>

Wickham, H., & Grolemund, G. (2023). *R for Data Science*. O'Reilly Media.

Pedregosa, F., Varoquaux, G., Gramfort, A., et al. (2011). *Scikit-learn: Machine Learning in Python*. *Journal of Machine Learning Research*, 12, 2825–2830.

Matplotlib Developers. (2024). *Matplotlib Visualization Library*. Available at: <https://matplotlib.org>

The Pandas Development Team. (2024). *pandas: Data Analysis and Manipulation Tool*. Available at: <https://pandas.pydata.org>

9. APPENDIX

APPENDIX A – Installation and Environment Setup

This section provides the steps for setting up the development environment used throughout the internship projects.

Operating System: macOS (Apple M4)

IDE & Tools: Visual Studio Code, RStudio, SQL Server Management Studio, and Terminal

Version Control: Git & GitHub (SSH-based configuration)

Python Environment:

venv virtual environment created via `python3 -m venv .venv`

Dependencies managed with `requirements.txt`

R Environment:

Required packages installed via `install.packages()` (e.g., *DBI*, *ggplot2*, *odbc*, *dplyr*, *dotenv*)

Database Connection:

SQL Server accessed via ODBC connection using environment variables stored in `.env` file (not shared publicly for security reasons).

APPENDIX B – Project Structure and Code Organization

Each project developed during the internship followed a modular and version-controlled structure hosted on GitHub.

Repositories included:

cu-veri-analizi – Forecasting academic publication trends (Prophet & Linear Regression)

basari-siniflandirma – Classification of academic performance using STRATEJIK_PLAN data

University-Units-Treemap – Hierarchical visualization of YÖKSİS data using Streamlit + Plotly

agaclandirma-tahminleme – Forecasting and regression analysis for research areas

rezervasyon-segmentasyon – Clustering-based segmentation of reservation datasets

Vesikalik-Kontrol-API – FastAPI application for photo validation

Streamlit-Project – Streamlit-based interactive dashboards (mock prototypes)

Simple-Linear-Regression – Baseline R model for publication trend comparison

Each repository included a `README.md`, `.gitignore`, and environment configuration files, ensuring reproducibility and consistent documentation.